

Computer Network Report

Branch/Class/Batch: AIDS/TE/B2

Name: Mangilal Malviya

Roll no: A49

CN ASSIGNMENT

Title: Client server communication using server programming.

Aim:

To write a Python program using socket programming to implement a simple client-server network architecture, where the client sends a text message to the server and the server receives and acknowledges the message.

Theory

Socket programming enables two devices or processes to communicate over a network. A socket acts as an endpoint for communication.

In a client-server architecture:

The server creates a socket, binds it to an IP address and port, and waits for client

connections.

The client creates a socket and connects to the server.

The client sends a message to the server.

The server receives the message and sends a response.

TCP (SOCK_STREAM) is used because it provides reliable, connection-oriented communication.

Working

Server side:

Create a TCP socket.

Bind it to 127.0.0.1 and port 5000.

Listen for a client connection.

Accept the client connection.

Receive the text message.

Display the received message.

Send an acknowledgment to the client.

Close the connection.

Client

side:

Create a TCP socket.

Connect to the server using its IP address and port.

Enter/send a text message.

Wait for the server's response.

Display the response.

Close the connection.

server program

```
import socket
```

```
server_socket = socket.socket(socket.AF_INET,  
socket.SOCK_STREAM)
```

```
host = '127.0.0.1'
```

```
port = 8080
```

```
server_socket.bind((host, port))
```

```
server_socket.listen(1)
```

```
print("Server is waiting for a connection...")
```

```
client_socket, client_address =
```

```
server_socket.accept()
print("Connected to:", client_address)
```

```
message = client_socket.recv(1024).decode()
print("Message received from client:", message)
```

```
client_socket.send("Message received  
successfully!".encode())
```

```
client_socket.close()
server_socket.close()
```

client program

```
import socket
```

```
client_socket = socket.socket(socket.AF_INET,  
socket.SOCK_STREAM)
```

```
host = '127.0.0.1'  
port = 5000
```

```
client_socket.connect((host, port))
```

```
message = "Hello Server! This is the client."  
client_socket.send(message.encode())
```

```
response = client_socket.recv(1024).decode()  
print("Server response:", response)
```

```
client_socket.close()
```

Output

server:

Server is waiting for a connection...

Connected to: ('127.0.0.1', 54321)

Message received from client: Hello Server!

This is the

Part 2

Title: Classification of Network Devices and Transmission Media

Aim

To write a Python program that takes a list of network devices such as Switch, Router, Bridge, Access Point and transmission media types, and generates a classification report explaining their OSI layer of operation and primary function in a network topology.

Theory

A computer network consists of various networking devices and transmission media that work together to enable communication.

The OSI (Open Systems Interconnection) model divides network communication into seven layers. Different networking devices primarily operate at different

layers.

Switch: Mainly operates at Layer 2 (Data Link) and forwards frames using MAC addresses.

Router: Mainly operates at Layer 3 (Network) and forwards packets between different networks using IP addresses.

Bridge: Operates at Layer 2 (Data Link) and connects or filters traffic between network segments.

Access Point: Primarily operates at Layer 2 (Data Link) and provides wireless connectivity to a LAN.

Transmission media: Such as twisted-pair cable, coaxial cable, and fiber-optic cable operate at Layer 1 (Physical) because they carry electrical or optical signals.

A classification program can store this information in a dictionary and use the input device/media name as a key to retrieve its corresponding layer and function.

Working /

Procedure

Create a dictionary containing network devices and transmission media.

Store the OSI layer and primary function for each item.

Create a list containing the devices and media to be classified.

Pass the list to a classification function.

The function searches for each item in the classification database.

If the item is found, its layer and primary function are displayed.

If an unknown item is entered, the program displays an Unknown status.

The final information is displayed as a classification report.

Program:

```
network_info = {  
    "Switch": {  
        "layer": "Layer 2 - Data Link",  
        "function": "Connects devices within a LAN  
and forwards frames using MAC addresses."
```

```
}  
"Router": {  
  "layer": "Layer 3 - Network",  
  "function": "Connects different networks and  
forwards packets using IP addresses."  
},  
"Bridge": {  
  "layer": "Layer 2 - Data Link",  
  "function": "Connects and filters traffic  
between LAN segments using MAC addresses."  
},  
"Access Point": {  
  "layer": "Layer 2 - Data Link",  
  "function": "Provides wireless access and  
connects wireless devices to a wired LAN."  
},  
"Twisted Pair Cable": {  
  "layer": "Layer 1 - Physical",  
  "function": "Transmits data using electrical  
signals through copper wires."  
},  
"Coaxial Cable": {  
  "layer": "Layer 1 - Physical",  
  "function": "Transmits data using electrical  
signals through a copper conductor."
```

```

},
"Fiber Optic Cable": {
    "layer": "Layer 1 - Physical",
    "function": "Transmits data using pulses of
light through optical fibers."
},
"Wireless": {
    "layer": "Layer 1 - Physical",
    "function": "Transmits data through radio
waves without physical cables."
}
}

```

```

def classification_report(items):
    print("\n===== NETWORK
CLASSIFICATION REPORT =====\n")

```

```

for item in items:
    if item in network_info:
        info = network_info[item]

```

```

    print("Device/Medium :", item)
    print("Layer :", info["layer"])
    print("Primary Role :", info["function"])

```

```
print("-" * 55)
else:
    print("Device/Medium :", item)
    print("Status : Unknown")
    print("-" * 55)
```

```
items = [
    "Switch",
    "Router",
    "Bridge",
    "Access Point",
    "Twisted Pair Cable",
    "Fiber Optic Cable"
]
```

```
classification_report(items)
```

Output

```
NETWORK CLASSIFICATION REPORT
Device/Medium : Switch
Layer : Layer 2 ~ Data Link
Primary Role : Connects devices within a LAN and forwards frames using MAC
```

addresses.

Device/Medium : Router

Layer : Layer 3 - Network

Primary Role : Connects different networks and forwards packets using IP addresses.

Device/Medium : Bridge

Layer : Layer 2 - Data Link

Primary Role : Connects and filters traffic between LAN segments using MAC addresses.

Device/Medium : Access Point

Layer : Layer 2 - Data Link

Primary Role : Provides wireless access and connects wireless devices to a wired LAN.

Device/Medium : Twisted Pair Cable

Layer : Layer 1 - Physical

Primary Role : Transmits data using electrical signals through copper wires.

Device/Medium : Fiber Optic Cable

Layer : Layer 1 - Physical

Primary Role : Transmits data using pulses of light through optical fibers.
